PORTFOLIO · HBM DIGITAL DESIGN (RTL DESIGN)

Mini DRAM Controller

Command-level FR-FCFS · 행 / 열 명령 버스 · Per-bank Refresh · RFM — Spec · RTL · 검증 · PPA

지원 직무	SK hynix · HBM Digital Design (RTL Design)
지원자	권용현 · 010-6408-8646 · kyongh0331@naver.com
저장소	`github.com/yhkwon-git/mini-dram-controller` · v4 commit `ab9d522` · v1 commit 6049e5a
사용 도구	SystemVerilog · Verilator 5.052 · Yosys 0.52 · sv2v · OpenSTA 3.1 (Nangate45) · Python

p.3–21(v0 · v1 상세)의 파형 · 커버리지 · 합성 수치는 commit 6049e5a 재실행 측정, 이후 fa08330 · 7e2d4b5 반영. p.2와 p.22–26(v2 – v4)은 RTL commit ab9d522, 분석 스크립트 322632f 기준.

00 Summary

v0 → v4: 단일 FSM에서 행 / 열 명령 버스 · Per-bank Refresh · RFM까지

같은 DRAM 모델 위에서 컨트롤러를 다섯 번 설계하고, 전환마다 latency · 처리량 · 면적 · 타이밍을 같은 TB와 같은 합성 흐름으로 비교

버전	구조와 바뀐 점	포화 row hit RD	fmax · 면적
v0	단일 FSM: 요청 하나를 PRE · ACT · RD/WR 순서로 처리	–	1,182 MHz · 1,208 μm²
v1	깊이 4 큐 + 뱅크 엔진 ×4 + 사이클당 명령 1개 arbiter	19.9%	343 MHz · 9,692 μm²
v2	명령 단위 FR-FCFS(age matrix), tREFI refresh + PREA, RD 4개 파이프라인, 타이밍 7종 추가	66.7%	357 MHz · 13,478 μm²
v3	읽기 · 쓰기 큐 8 + 8, posted write · 포워딩, 쓰기 묶음, page 정책 3 · 주소 매핑 4	66.7%	337 MHz · 18,600 μm²
v4	행 · 열 명령 버스, 뱅크 그룹(tCCD · tRRD · tWTR S/L), per-bank refresh, RFM	**80.7%**	**348 MHz** · 18,912 μm²

포화: 요청 간격 0, back-pressure 없음, seed 1, 5,000건. v1 처리량은 2eabd33, 면적 · 타이밍은 fa08330. v4부터 장치에 뱅크 그룹이 있어 같은 장치의 v3 방식 빌드(bus1_refab)는 63.3%다. 면적 · fmax는 Nangate45 typical, mem_ctrl 단독.

80.7%

포화 row hit 읽기, 데이터 버스 사용률 · 같은 장치의 v3 방식 63.3% (+27%)

1,200 / 1,200

랜덤 회귀 PASS · 600만 요청 · 빌드 16 · 규칙 위반 0 · 기능 bin 32/32

98.7 · 98.2%

컨트롤러 Line · Branch 커버리지 (Toggle 93.1%), directed + 랜덤

348 MHz

Nangate45 최소 주기 2.874 ns · 18,912 μm² · FF 1,616 (v3 337 MHz)

가장 큰 손잡이: per-bank refresh

all-bank로 바꾸면 포화 평균 −16%, 면적 차이는 2%. 버스 분리와 함께일 때 row hit 63.3 → 80.7%

타이밍 클로저: 208 → 337 MHz (v3)

수락 경로를 스케줄러에서 떼고, 후보 집합을 합치고, 선택을 one-hot 그대로 유지. v4는 기능을 더하고도 348 MHz

검증: 독립 체커 + 랜덤 회귀

DRAM 핀만 보는 bind 체커(타이밍 18 · 프로토콜 10종)와 class 랜덤 TB. 회귀 1,200회 위반 0, TB race 1건 디버그

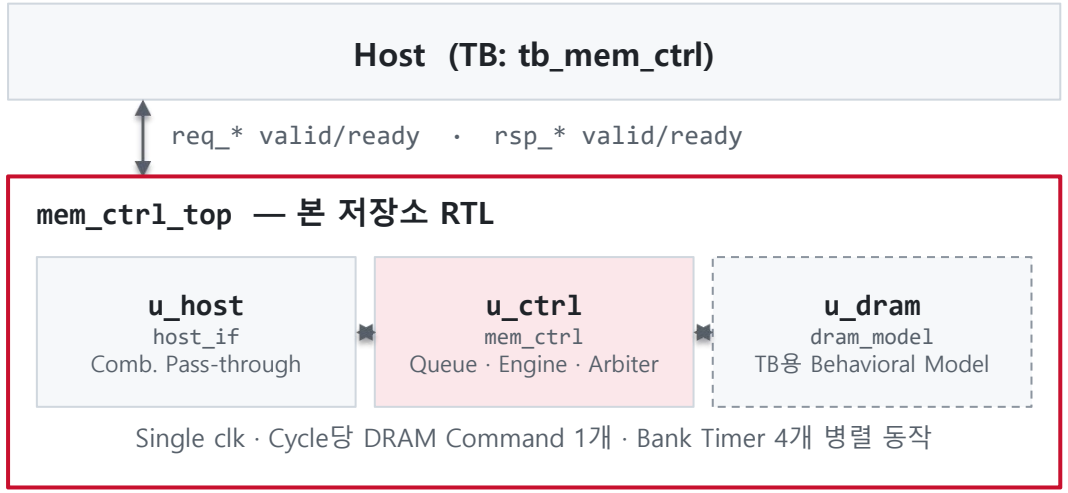
읽는 순서 **p.3–21** v0 · v1 상세 (스펙, 구조, RTL, 파형, 디버그, PPA) → **p.22–26** v2 – v4 (구조, 검증 인프라, 성능, 트래픽 × 기능 분석, PPA)

Bank 상태에 따른 Access Latency 편차와 Bank 병렬화 필요성

DRAM Controller의 핵심: Command 간 Timing Constraint를 Cycle 단위로 준수하면서, 대기 구간을 타 Bank 동작으로 Hiding

개발 배경

- 동일 Bank 0 기준 Page Hit RD 6클럭, Row Conflict RD 14클럭 (재실행 측정). PRE · ACT 대기 구간을 타 Bank Command로 채우는 것이 Throughput의 핵심.
- v0는 요청을 순차 처리하는 단일 FSM 구조. v1(`6049e5a`)은 Depth-4 Queue + Bank별 Timing Engine ×4 + Cycle당 1 Command Arbiter 구조로 개선 — 본 자료는 구조 전환에 따른 Trade-off를 정량 분석.
- Timing은 자체 축소 모델 적용 (단위 클럭: tRCD 3 · tRP 3 · tRAS 6 · tRC 9 · tRFC 12 · tREFI 64). JEDEC 수치가 아닌 구조 · 규칙 검증 목적의 Study.



Scope — docs/spec.md

`host_if` → `mem_ctrl` → `dram_model`, Single Clock 동기 설계 · Async Reset. 실제 PHY · Multi-Rank · ECC · Sign-off PPA는 범위 외.

v0 (`0cfe062`) IDLE에서 요청 1개 Latch → PRE/ACT/RD/WR 순차 수행
v1 (`6049e5a`) Queue 수신 → IDLE Bank로 Dispatch → Bank Engine 병렬 수행

5 파일 · 742줄
RTL 5파일 626줄 (주석 · 빈 줄 제외) · `mem_ctrl` 355줄 · TB 241줄

PASS · 위반 0
저장소 TB 4개 Scenario · 요청 18개 · VCD 기반 Timing Rule 7종 Check

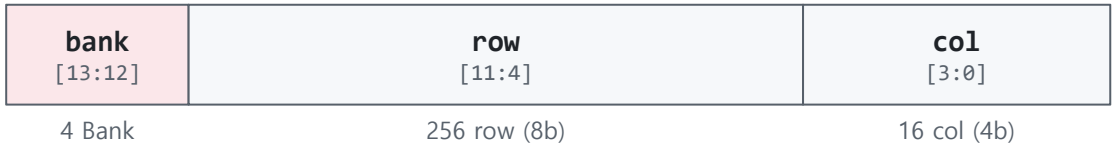
2.06× Burst
4-Bank Write 4건: 첫 Accept → 마지막 Response 17클럭 (v0 35클럭)

373 MHz
Area 9,819 μm² (Nangate45 typical, Routing 미포함) · `mem_ctrl` 기준 참고값

Spec 정의: Address Map · Timing Parameter · Command Set

docs/spec.md와 rtl/dram_pkg.sv를 단일 Source로 관리 · 우측 표의 '구현 위치'는 각 Timing Rule을 보장하는 RTL Logic

Address Map (14b): Bank · Row · Column



4 × 256 × 16 = 16,384 Word × 32b (64 KiB) · TB: `make_addr(b, r, c) = {b, r, c}`

Command Set dram_cmd_e (3b) — Model 동작

Command	Code	dram_model 동작
NOP	000	No Operation
ACT	001	bank_open ← 1, bank_row ← row · Open Bank에 다른 Row 시 access_err
RD	010	tCL 2 후 data_valid · rdata · Closed/다른 Row 시 access_err (Data 0)
WR	011	Row 일치 시 Memory Write · 불일치 시 access_err (Write 생략)
PRE	100	bank_open ← 0 (Single Bank)
REF	101	All Bank Close · tRFC 12클럭 동안 busy

Host: `req` valid/ready · `req_write` 1 = WR · Response는 요청 순서(In-order)로 반환, WR도 Response 발생 (Data 0) · `rsp_err`는 v1에서 0 고정

Timing Parameter (클럭) — RTL 구현 위치

파라미터	값	정의	구현 위치 (RTL)
tRCD	3	ACT → RD/WR	`b_timer` · ST_ACT_WAIT
tRP	3	PRE → ACT	`b_timer` · ST_PRE_WAIT
tRAS	6	ACT → PRE Min	`ras_rem` → want_pre
tRC	9	Same Bank ACT → ACT	`rc_rem` → want_act
tWR	2	Write Recovery	`b_timer` · ST_WR_WAIT
tCL	2	RD → data_valid	Model `rd_cnt`
tRFC	12	REF busy	Model `rfc_cnt` → busy
tREFI	64	Refresh 주기	`refi_cnt` → ref_due

문서 ↔ RTL 정합 분석 시점(6049e5a)의 spec §4/§5 · scheduler-throughput.md는 단일 FSM(본 자료의 v0) 기준으로 기술 → v1(Queue Accept · Bank별 State) 기준으로 갱신 완료 (d3061ca). `ST_REF` · `ST_REF_WAIT` · `ST_ERROR`는 Reserved (REF는 `ref_active` Flag로 처리). 본 자료의 수치는 모두 v1 RTL 재실행 측정값.

Architecture 비교: 단일 FSM(v0) vs. Queue + Bank Engine(v1)

동일 TB · 동일 측정 Script 기준 비교 — v0: 0cfe062/64e73e0 mem_ctrl, v1: 6049e5a

	v0 단일 FSM	v1 Queue + Bank Engine ×4
Accept 조건	ST_IDLE에서 요청 1개	Queue 빈 Slot 존재 시 (Depth 4) · REF 대기 중 차단
State 관리	State 1개 · Timer 1개	Bank별 `b_state` · `b_timer` + Open Row Shadow
Command Issue	Wait 종료 Cycle에 즉시 Issue	Bank `want_*` → Arbiter PRE > ACT > WR > RD, Cycle당 1개
Response 순서	단일 처리로 자연 In-order	Tag 16개 Response Buffer → `tag_retire` 순서로 반환
Refresh	IDLE에서 REF 우선	`ref_due` → Accept 차단 → All Bank IDLE · Queue Empty 시 REF
4-Bank Burst	35클럭	**17클럭 (2.06×)**
Hit Latency	4클럭	6클럭 — 구조 Overhead +2 (p.18)
Area · F/F	1,208 μm² · 131	9,819 μm² · 1,072 (8.1×, p.21)

작성 당시(v1) High Speed Link Lite의 MC는 Queue Head만 Issue하는 In-order 구조였다. 지금은 이 저장소의 v4 컨트롤러를 DFI 형태로 그대로 쓴다(HSL 0193c04).

Open-page Policy

Access 후에도 Row Open 유지. 동일 Row는 즉시 RD/WR, 타 Row는 PRE → ACT. Close 조건은 Conflict · REF

Oldest-tag Dispatch

Queue 내 Target Bank가 IDLE인 Entry 중 최고령 Tag 우선 Dispatch. `q_tag - tag_retire`로 Wrap-around 시에도 Age 비교 유지

Drain-then-REF

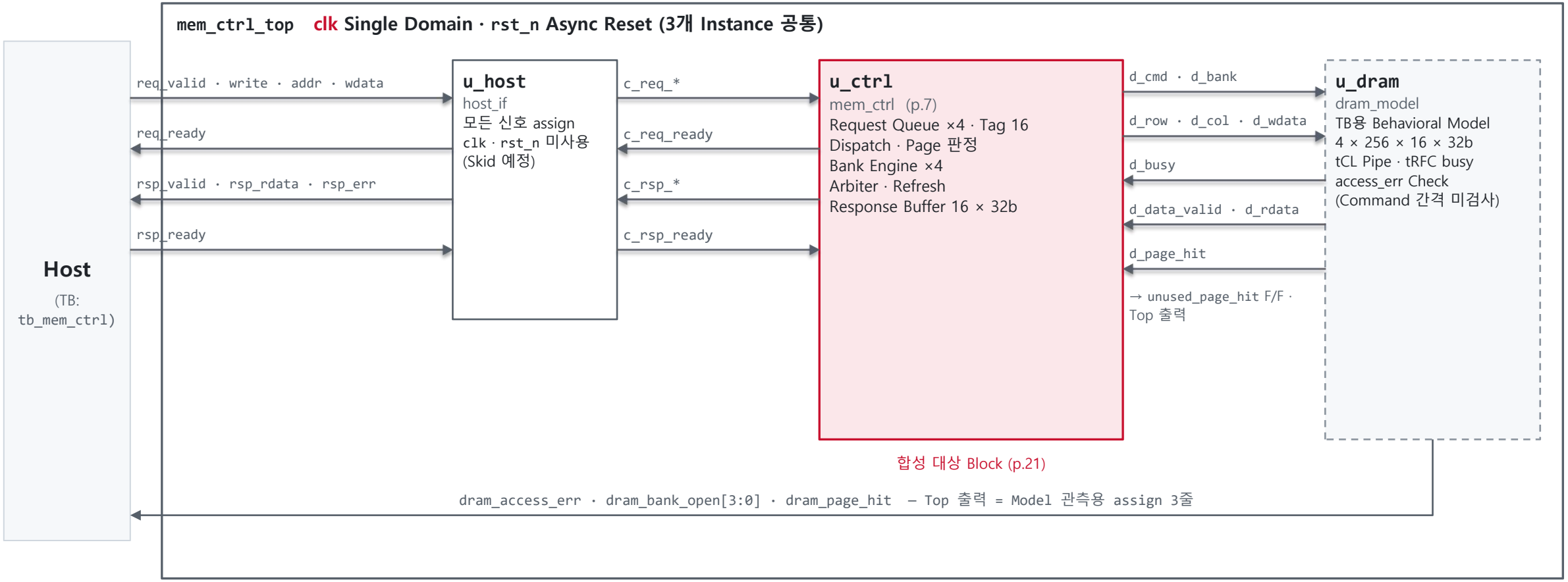
`ref_due` 시 신규 Accept 차단, 진행 중 Bank · Queue Drain 후 REF. 구조는 단순하나 REF 시 Open Row 전체 Close

Shadow Open-row

Controller가 `bank_open` · `bank_row` 및 tRAS/tRC 잔여 Cycle을 자체 관리. Model의 `page_hit` 출력 미사용

Top-level 구조: host_if → mem_ctrl → dram_model

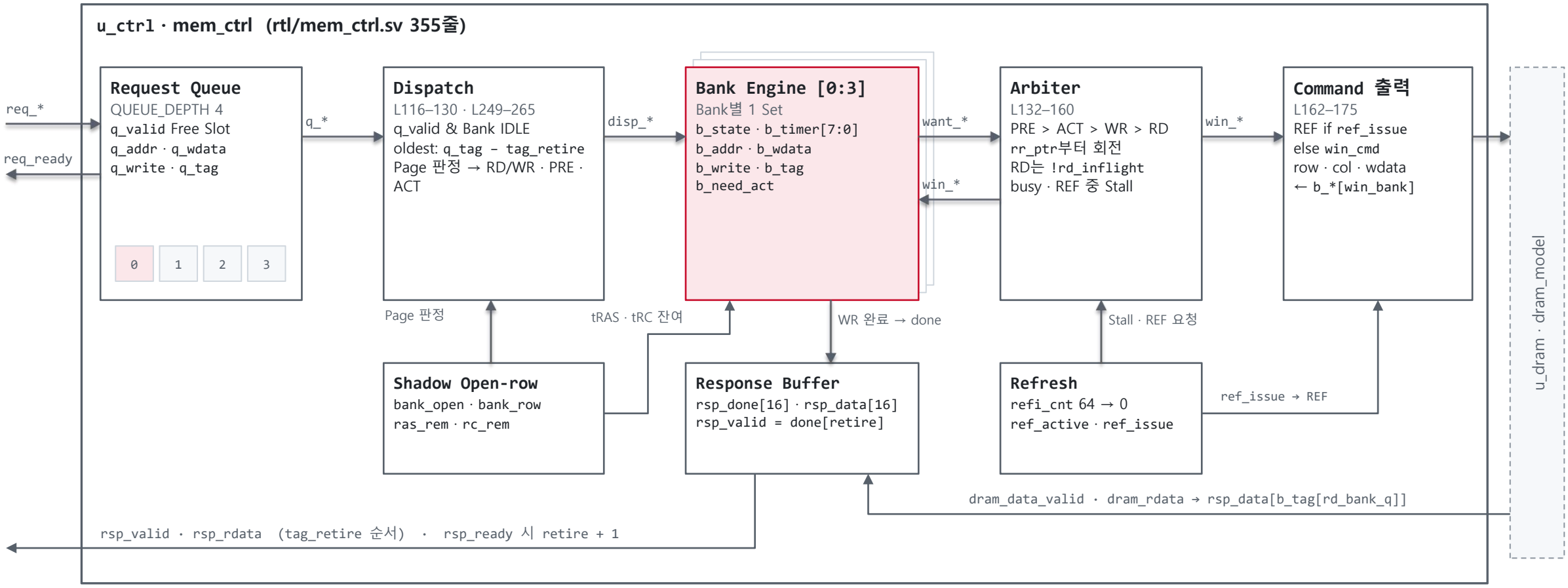
단일 Clock 동기 설계 · rtl/mem_ctrl_top.sv 기준 Instance · Port · Net 명칭 동일 표기



Legend `req_addr[13:0]` = {bank, row, col} · `req_wdata` · `rsp_rdata` 32b · `d_cmd` = dram_cmd_e 3b · `c_*`는 Host측 신호와 1:1 (host_if L39–48)
Hit 판정은 Controller의 Shadow `bank_open` / `bank_row`로 수행 — Model의 `page_hit`은 관측용. 실선 = RTL, 점선 = TB Model

mem_ctrl 구조: Queue → Dispatch → Bank Engine → Arbiter

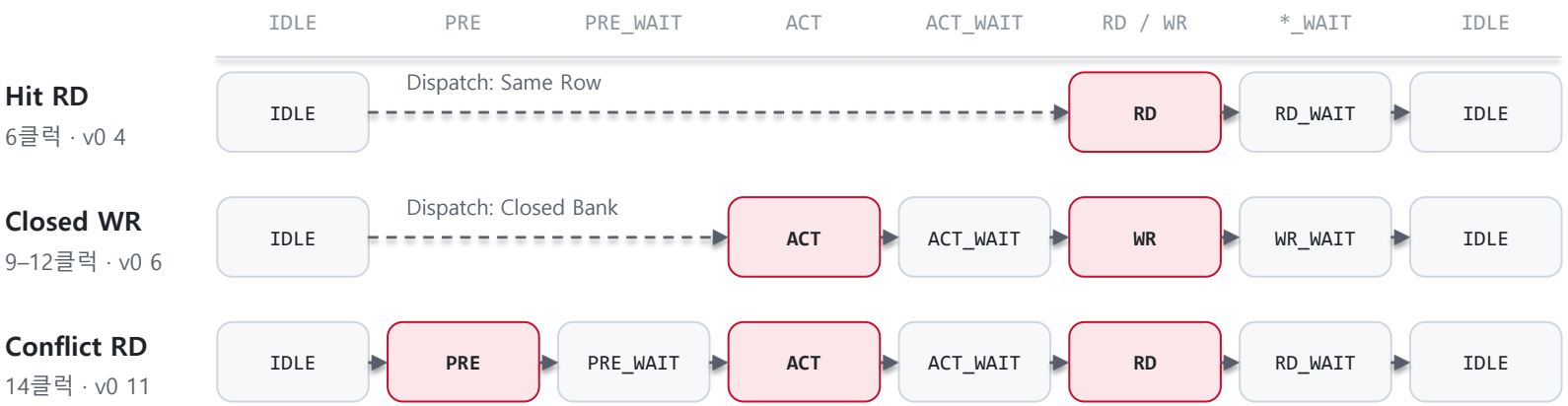
단일 always_comb에서 want_* · ref_issue · req_ready · disp_* · win_* 생성, 단일 always_ff에서 전체 State Update



`req_ready = free_found && !ref_due && !ref_active` (L113, 6049e5a 기준) — Response Buffer 여유 미확인 → p.19 Tag Wrap-around 이슈 원인. Critical Path: `tag_retire` → Dispatch Age 비교 → `b_state` (p.21)

Bank Engine State Transition 경로별 Latency 분석

열 = b_state, 행 = Dispatch 시점 Page 판정 · 우측 수치는 재실행 측정 Accept → Response 클럭 (v0 동일 TB)



굵은 Box = Arbiter에 Command 요청 State (`want_*`) · 점선 = Skip State

Transition	조건	동작
PRE → PRE_WAIT	`want_pre = ras_rem == 0` · Arbitration Grant	PRE Issue · `b_timer ← tRP-1` · bank_open ← 0
PRE_WAIT → ACT	`b_timer == 0` & `b_need_act`	(need_act = 0 시 IDLE — TB 미도달)
ACT → ACT_WAIT	`want_act = rc_rem == 0` · Arbitration Grant	ACT Issue · `b_timer ← tRCD-1` · tRAS/tRC Load
ACT_WAIT → RD/WR	`b_timer == 0`	State만 전환 → 다음 Cycle Arbitration (p.18 +1)
RD → RD_WAIT → IDLE	Grant & `!rd_inflight` → `dram_data_valid`	`rsp_data[b_tag] ← rdata` · rd_inflight ← 0
WR → WR_WAIT → IDLE	Grant → `b_timer == 0`	`b_timer ← tWR-1` → `rsp_done[b_tag] ← 1`

`u_dram · dram_model`

TB용 Behavioral Model (rtl/dram_model.sv)
Model측 `bank_open` · `bank_row` ×4
RD: tCL 2 Pipe → `data_valid` · `rdata`
REF: All Bank Close, tRFC 12클럭 동안 busy
`access_err`: Open Bank에 다른 Row ACT (L110), Closed/다른 Row RD · WR (L122 · L132)
Command 간격(tRCD 등) 미검사

- Engine 4개가 timer · want를 독립 계산, Command Bus만 공유 → p.16에서 8클럭 연속 Issue
- `ST_REF` · `ST_REF_WAIT` · `ST_ERROR`는 enum에만 존재 (Reserved)
- Write Hit 경로 (Dispatch → ST_WR, L259)는 TB 미도달 (p.14)
- Model이 Timing 미검사 → 별도 VCD Checker로 Violation 0 확인 (p.13 · 14)

Request Accept 및 Oldest-first Dispatch 구조

req_ready는 Queue · Refresh 상태 기준 판단 · Busy Bank 요청은 Queue 대기, 타 Bank 요청 우선 Dispatch 가능

rtl/mem_ctrl.sv L113–114, L116–130

```
113  req_ready = free_found && !ref_due && !ref_active;                          ①
114  q_push    = req_valid && req_ready;

116  // Prefer oldest tag among ready-to-dispatch entries
117  disp_valid = 1'b0;
118  disp_bank  = '0;
119  disp_slot  = '0;
120  if (!ref_active && !ref_issue) begin                                        ②
121    for (bi = 0; bi < QUEUE_DEPTH; bi++) begin
122      if (q_valid[bi] && (b_state[addr_bank(q_addr[bi])] == ST_IDLE)) begin   ③
123        if (!disp_valid || (q_tag[bi] - tag_retire) < (q_tag[disp_slot] - tag_retire)) begin  ④
124          disp_valid = 1'b1;
125          disp_bank  = addr_bank(q_addr[bi]);
126          disp_slot  = bi[QUEUE_AW-1:0];
127        end
128      end
129    end
130  end
```

rtl/mem_ctrl.sv L249–265

```
249  // Dispatch
250  if (disp_valid) begin
251    q_valid[disp_slot] <= 1'b0;                                               ⑤
252    b_addr[disp_bank]  <= q_addr[disp_slot];
253    b_wdata[disp_bank] <= q_wdata[disp_slot];
254    b_write[disp_bank] <= q_write[disp_slot];
255    b_tag[disp_bank]   <= q_tag[disp_slot];
256    b_need_act[disp_bank] <= 1'b0;
257    if (bank_open[disp_bank] &&                                               ⑥
258        (bank_row[disp_bank] == addr_row(q_addr[disp_slot])))
259      b_state[disp_bank] <= q_write[disp_slot] ? ST_WR : ST_RD;
260    else if (bank_open[disp_bank]) begin                                      ⑦
261      b_need_act[disp_bank] <= 1'b1;
262      b_state[disp_bank]    <= ST_PRE;
263    end else
264      b_state[disp_bank] <= ST_ACT;                                           ⑧
265  end
```

① `req_ready` = Free Slot & !ref_due & !ref_active — Response Buffer 여유 미확인 (p.19 이슈 원인, 이후 수정)

② REF Issue 또는 진행 중 Dispatch 정지 — REF 전 Queue Drain 보장

③ Target Bank가 ST_IDLE인 Entry만 후보 → Busy Bank 요청은 대기, 타 Bank 요청 선행 (TB Queue 점유 최대 1로 미관찰)

④ `q_tag - tag_retire` (4b Modular)로 Age 비교 → Tag Wrap-around 시에도 Oldest 유지

⑤ Dispatch Cycle에 Slot 해제, Address · Data · Tag를 Bank Engine `b_*`로 복사

⑥ Shadow bank_open / bank_row로 Hit 판정 → 즉시 ST_RD / ST_WR (ACT 생략)

⑦ Open 상태의 다른 Row → ST_PRE + `b_need_act` → PRE_WAIT 후 자동 ACT

⑧ Closed Bank → ST_ACT. 3가지 경로 = p.8 Hit / Conflict / Closed

Command Arbitration 및 Issue 구조

want_* (Combinational) → PRE > ACT > WR > RD Priority + rr_ptr Round-robin → Issue Cycle에 Timer Preload · Shadow Update

rtl/mem_ctrl.sv L92–97, L135–141, L156–157

```
92  case (b_state[bi])
93    ST_PRE: want_pre[bi] = (ras_rem[bi] == 0);                          (1)
94    ST_ACT: want_act[bi] = (rc_rem[bi] == 0);
95    ST_RD:  want_rd[bi]  = 1'b1;
96    ST_WR:  want_wr[bi]  = 1'b1;
97    default: ;
      ---------------------------------------------------------------------
135 if (!dram_busy && !ref_active && !ref_issue) begin                    (2)
136   for (bi = 0; bi < NUM_BANKS; bi++) begin
137     automatic int b = (int'(rr_ptr) + bi) % NUM_BANKS;                 (3)
138     if (!win_valid && want_pre[b]) begin                               (4)
139       win_valid = 1'b1; win_bank = b[$clog2(NUM_BANKS)-1:0]; win_cmd = CMD_PRE;
140     end
141   end
      ---------------------------------------------------------------------
156 if (!win_valid && want_rd[b] && !rd_inflight) begin                     (5)
157   win_valid = 1'b1; win_bank = b[$clog2(NUM_BANKS)-1:0]; win_cmd = CMD_RD;
```

rtl/mem_ctrl.sv L285–296, L323–324

```
285 CMD_PRE: begin
286   bank_open[win_bank] <= 1'b0;
287   b_timer[win_bank]   <= 8'(tRP > 0 ? tRP - 1 : 0);
288   b_state[win_bank]   <= ST_PRE_WAIT;
289 end
290 CMD_ACT: begin
291   bank_open[win_bank] <= 1'b1;                                         (6)
292   bank_row[win_bank]  <= addr_row(b_addr[win_bank]);
293   ras_rem[win_bank]   <= tRAS[$bits(ras_rem[0])-1:0];
294   rc_rem[win_bank]    <= tRC[$bits(rc_rem[0])-1:0];
295   b_timer[win_bank]   <= 8'(tRCD > 0 ? tRCD - 1 : 0);                  (7)
296   b_state[win_bank]   <= ST_ACT_WAIT;
      ---------------------------------------------------------------------
323 if (b_state[i] == ST_ACT_WAIT && b_timer[i] == 0)                      (8)
324   b_state[i] <= b_write[i] ? ST_WR : ST_RD;
```

1. want_pre는 tRAS 잔여 0, want_act는 tRC 잔여 0일 때만 Assert — Bank별 독립 Counter
2. DRAM busy 또는 REF 진행 중 Command Issue 정지
3. rr_ptr부터 4-Bank 검사 → 동일 Command 간 Round-robin (Issue 후 rr_ptr ← win_bank + 1)
4. PRE → ACT → WR → RD 순 Loop — Row Command 선행 Issue로 타 Bank tRP/tRCD Overlap
5. RD는 !rd_inflight일 때만 → Read Pipe 1개, RD 최대 4클럭당 1회 (p.18)
6. ACT Cycle에 Shadow bank_open · bank_row 및 tRAS · tRC 잔여 Cycle 동시 Load
7. b_timer ← tRCD − 1: Wait 종료 Cycle 즉시 Issue하던 v0 기준값. v1 실제 간격은 tRCD + 1
8. ACT_WAIT 종료 시 State만 ST_RD/WR 전환 → 실제 RD/WR는 다음 Cycle Arbiter에서 Issue

Drain-then-REF Refresh 정책 및 Tag 기반 In-order Response

refi_cnt = 0 시 신규 Accept 차단 후 진행 중 Transaction 완료 대기 · 완료 Data는 Buffer 저장, 반환은 tag_retire 순서

rtl/mem_ctrl.sv L101–102, L218–224, L227–230

```
101  ref_due   = (refi_cnt == 0);                                                    (1)
102  ref_issue = ref_due && !ref_active && all_banks_idle && (q_count == 0) && !dram_busy;  (2)

218  if (ref_active) begin
219    if (!dram_busy) begin                                                         (3)
220      ref_active <= 1'b0;
221      refi_cnt   <= tREFI[$bits(refi_cnt)-1:0];
222    end
223  end else if (refi_cnt != 0) begin
224    refi_cnt <= refi_cnt - 1'b1;

227  if (ref_issue) begin
228    ref_active <= 1'b1;
229    for (int i = 0; i < NUM_BANKS; i++) begin
230      bank_open[i] <= 1'b0;                                                       (4)
```

rtl/mem_ctrl.sv L178–179, L327–329, L335–337, L342–345

```
178  rsp_valid = rsp_done[tag_retire];                                               (5)
179  rsp_rdata = rsp_data[tag_retire];

327  if (b_state[i] == ST_WR_WAIT && b_timer[i] == 0) begin
328    rsp_done[b_tag[i]] <= 1'b1;
329    rsp_data[b_tag[i]] <= '0;                                                     (6)

335  if (rd_inflight && dram_data_valid) begin
336    rsp_done[b_tag[rd_bank_q]] <= 1'b1;
337    rsp_data[b_tag[rd_bank_q]] <= dram_rdata;                                     (7)

342  if (rsp_valid && rsp_ready) begin                                               (8)
343    rsp_done[tag_retire] <= 1'b0;
344    tag_retire <= tag_retire + 1'b1;
345  end
```

1. `ref_due = refi_cnt == 0` → 동일 Cycle부터 req_ready 0 (p.9 ①)
2. REF 조건: All Bank IDLE & Queue Empty & DRAM idle — 첫 REF는 진행 중 WR로 6클럭 대기 (p.17)
3. busy 해제 Cycle에 `ref_active` Clear, refi_cnt 64 Reload
4. REF 시 Model의 All Bank Close에 맞춰 Shadow도 Close → REF 직후 동일 Row는 Hit가 아닌 Closed
5. `rsp_valid = rsp_done[tag_retire]` — 최고령 요청 완료 시 Response. 후속 요청 선완료 시 Buffer 대기
6. WR도 Response 발생 (Data 0) → Host는 요청 수 = Response 수로 관리 가능
7. RD Data를 `b_tag[rd_bank_q]` Entry에 저장 — Entry 점유 여부 미확인 → Tag Wrap-around 시 Overwrite (p.19)
8. Handshake 시 done Bit Clear 및 `tag_retire + 1`

DRAM Model Protocol Check 및 Timing Parameter 관리

dram_model은 State · Data · tCL/tRFC만 Modeling (Command 간격 미검사) — 별도 VCD Checker로 보완

rtl/dram_model.sv L108–115, L120–125, L130–135

```
108  CMD_ACT: begin
109    // Opening a new row on an already-open bank without PRE is a conflict
110    if (bank_open[bank] && bank_row[bank] != row)                    ①
111      access_err <= 1'b1;
112    bank_open[bank] <= 1'b1;
113    bank_row[bank]  <= row;
114    page_hit        <= 1'b0;
115  end
     ------------------------------------------------------------------
120  CMD_RD: begin
121    page_hit   <= bank_open[bank] && (bank_row[bank] == row);
122    access_err <= ~(bank_open[bank] && (bank_row[bank] == row));     ②
123    rd_active  <= 1'b1;
124    rd_was_hit <= bank_open[bank] && (bank_row[bank] == row);
125    rd_cnt     <= tCL[$bits(rd_cnt)-1:0];                            ③
     ------------------------------------------------------------------
130  CMD_WR: begin
131    page_hit   <= bank_open[bank] && (bank_row[bank] == row);
132    access_err <= ~(bank_open[bank] && (bank_row[bank] == row));
133    if (bank_open[bank] && (bank_row[bank] == row))                  ④
134      mem[mem_index(bank, row, col)] <= wdata;
135  end
```

Coverage dram_model Branch 18개 중 3개 미도달 — L110 if (ACT 충돌), L133 else (Illegal WR), L93 else (Miss RD). Model Error 경로 검출 이력이 없어 `access_err count = 0`은 '위반 미발생'까지만 의미 (p.20 E).

rtl/dram_pkg.sv L20–27

```
20  localparam int tRCD = 3;   // ACT -> RD/WR                    ⑤
21  localparam int tRP  = 3;   // PRE -> ACT
22  localparam int tRAS = 6;   // ACT -> PRE (min)
23  localparam int tRC  = 9;   // ACT -> ACT same bank (≈ tRAS+tRP) ⑥
24  localparam int tRFC = 12;  // REF busy window
25  localparam int tCL  = 2;   // RD -> data_valid
26  localparam int tWR  = 2;   // WR recovery before PRE
27  localparam int tREFI = 64; // refresh opportunity period       ⑦
```

① Open Bank에 다른 Row ACT (PRE 누락) → `access_err`

② Closed Bank 또는 다른 Row로 RD 시 `access_err` — RD/WR마다 재판정

③ tCL Count 후 `data_valid` 1클럭 → Controller RD_WAIT 종료 조건

④ Illegal WR는 Memory 미반영 → Data 비교로도 검출 가능

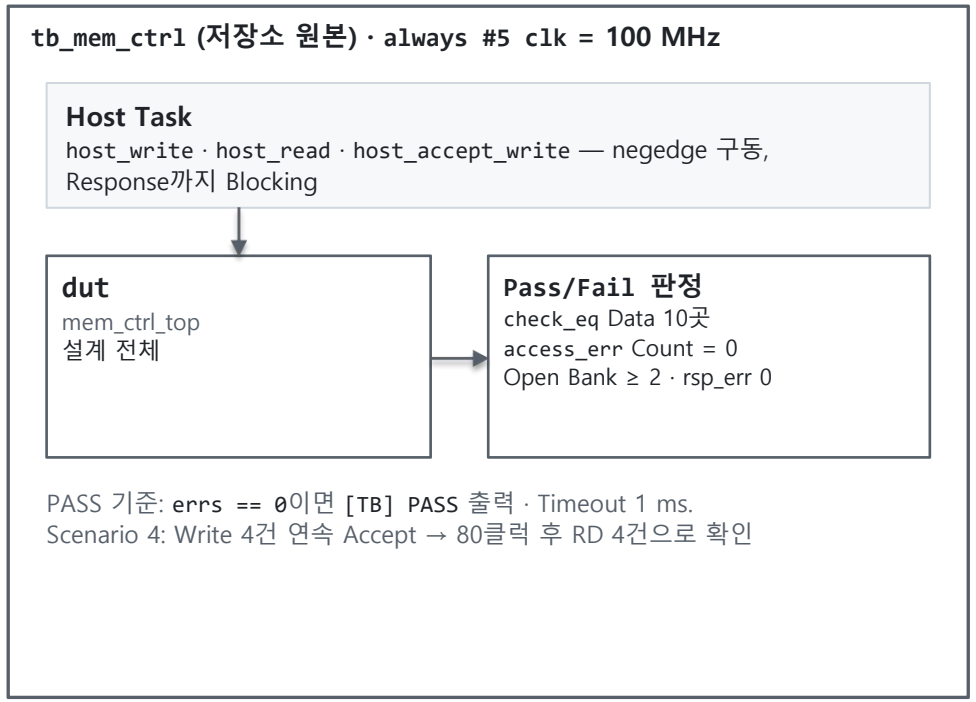
⑤ tRCD · tRP · tRAS → Bank Engine `b_timer` · `ras_rem`에서 사용

⑥ tRC 9 = tRAS 6 + tRP 3 (Spec 일치) · 측정 Min tRC 23, Violation 0

⑦ tREFI 64 · tRFC 12 → 문서 추정 REF 비중 ≈19%, 측정값 18.3%

Verification 전략: Directed TB 및 VCD 기반 추가 분석 환경

저장소 tb_mem_ctrl(6049e5a)을 원본 그대로 실행하고, VCD 분석 · 실험용 TB를 별도 Layer로 구성 — 실험 Code는 저장소 외부 관리



분석 Layer (별도 실험 환경)

`tb_pf_top`: tb_mem_ctrl Instance + `$dumpvars` → full.vcd
`tb_pf_backpressure`: rsp_ready 150클럭 0 + Read 20건 (p.19)
`analyze.py`: VCD → Command 간격 · Latency 분류 · Engine 동시성

Scenario	Stimulus	Check
1 Page Hit	WR b0 r11 → 동일 Address RD	A5A5_5A5A 일치
2 Page Miss	b0 r22 WR · RD → r11 재RD	신규 Row · 기존 Row Data 모두 일치
3 Multi-Bank	b1 r33 WR · RD, b0 r44 Conflict, b1 재확인	b0 Row 변경 후 b1 Data 유지
4 4-Bank Burst	b0–b3 WR 4건 연속 Accept → RD 4건 확인	Data 4건 · Open Bank ≥ 2
공통	전 구간	access_err 0 · rsp_err 0

추가 검증 항목 (저장소 TB 미포함)

항목	방법	결과
Timing Rule 7종	VCD Command를 Bank별 Pairing하여 tRCD · tRP · tRAS · tRC · tWR · tCL · tRFC Min 간격 측정	p.14
Latency 분류	요청별 Hit / Closed / Conflict 분류 및 Accept → Response 클럭	p.15–18
v0 A/B	v0 mem_ctrl (64e73e0) + HEAD Model · TB로 동일 측정	p.5 · 18
Back-pressure	coverage.md Gap — rsp_ready Block 상태에서 Read 20건	p.19
Coverage · 합성	Verilator line · branch · toggle, Yosys + OpenSTA	p.14 · 21

Verification 결과 요약: PASS · Timing Violation 0

mem_ctrl Line 94.3% / Branch 91.9% · Verilator 5.052 재실행 (312클럭) · CMD: ACT 14 · PRE 4 · RD 10 · WR 8 · REF 4

sim/regress.log (make sim 동일 Flag)

```
=== SCENARIO 1: page hit WR→RD ===
hit RD OK (0xa5a55a5a)
=== SCENARIO 2: page miss (bank0 row change) ===
miss-new RD OK (0xbeef0002)
miss-old RD OK (0xa5a55a5a)
=== SCENARIO 3: multi-bank + bank0 conflict ===
bank1 RD OK (0xc0de0001)
bank0 conflict RD OK (0xd00d0004)
bank1 survived OK (0xc0de0001)
=== SCENARIO 4: queue depth — burst accept 4 banks ===
peak banks open during burst=4
q bank0 OK (0x10000000)
q bank1 OK (0x20000000)
q bank2 OK (0x30000000)
q bank3 OK (0x40000000)
access_err count=0 OK
PASS — hit/miss/multi-bank/queue-parallel OK
```

- `verilator -Wall --timing --sv --binary` (Makefile `sim`과 동일) · Lint: Error 0, UNUSEDSIGNAL Warning 9개 (host_if `clk`/`rst_n`, mem_ctrl Loop 변수 `b` 상위 Bit 등)
- Waveform은 `--trace` Wrapper, Coverage는 `--coverage-line --coverage-toggle` 별도 Build · cocotb 경로 미실행

Coverage (Verilator)

Module	Line	Branch	Toggle
mem_ctrl	94.3%	91.9%	51.2%
dram_model	100.0%	83.3%	75.7%
host_if	–	–	81.5%
mem_ctrl_top	–	–	82.1%
Total	95.8%	90.2%	63.7%

Timing Rule (VCD Checker)

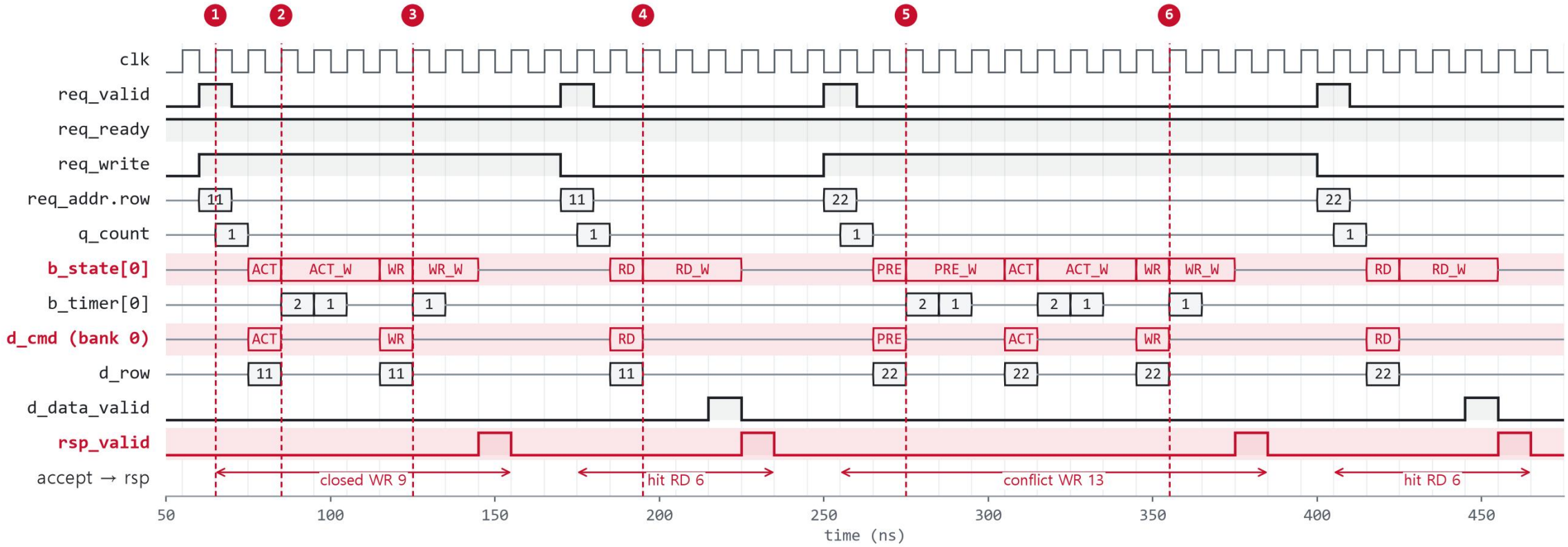
Rule	Spec	측정 Min	Count	위반
tRCD	3	4	18	0
tRP	3	4	4	0
tRAS	6	19	4	0
tRC	9	23	4	0
tWR	2	15	3	0
tCL	2	2	10	0
tRFC	12	16	3	0

tWR = WR → 동일 Bank PRE, tRFC = REF → 다음 Command. 음영: Spec 대비 +1클럭 (p.18)

Coverage Hole 및 보완 계획

- Queue 점유 최대 1 — Slot 1–3 미사용. Response까지 대기하는 Blocking Task 구조로 Depth 4 미검증 (분석 당시 coverage.md 'Depth-4 accept' 기술과 불일치)
- Write Hit (L259) · Push/Dispatch 동시 (L271) · PRE_WAIT → IDLE (L315) · 후보 간 Age 비교 (L123 else) 미도달
- Model Error Branch 3개 미도달 → access_err 경로 Negative Test 부재
- Toggle 51.2%: `rsp_data` 16 × 32b 등 Wide Register 대부분 고정값
- L287 · 295 · 299 cond_else, L307 default는 상수 · Unreachable Branch
- coverage.md Gap 'Backpressure'를 p.19 실험으로 검증 → 이슈 발견 · 수정 반영

Page Policy별 Latency: Closed 9 · Hit 6 · Conflict 13클럭



1 65 ns WR b0 Row 11 Accept · q_count 1. Bank Closed → Closed 경로

2 85 ns ACT — Accept 후 2클럭 (Queue 등록 → Dispatch → Issue)

3 125 ns WR — ACT 후 4클럭 = tRCD 3 + 1 (Wait 종료 다음 Cycle Issue)

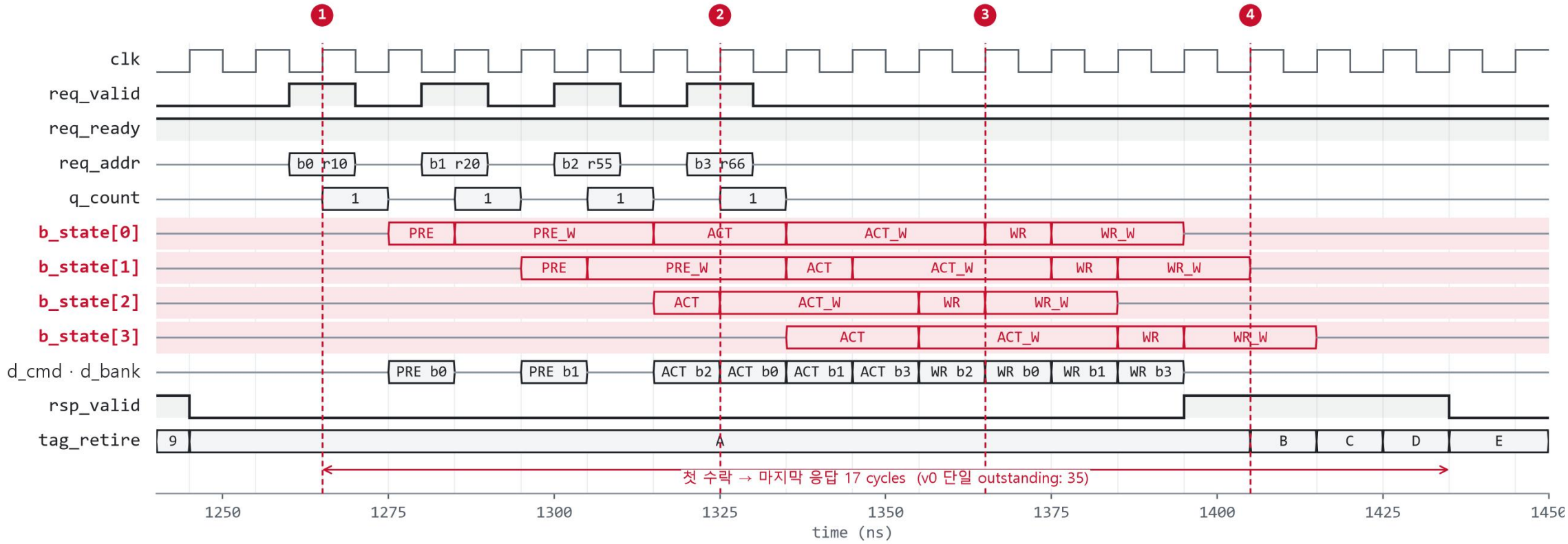
4 195 ns 동일 Row RD → ACT 없이 즉시 RD, Hit 6클럭

5 275 ns Row 22 WR → Open Row 11과 Conflict, PRE 선행 (`b_need_act`)

6 355 ns PRE → ACT 4 · ACT → WR 4클럭 → Conflict 13클럭

tb_mem_ctrl Scenario 1 · 2 (Bank 0), 50–475 ns · 저장소 TB를 `tb_pf_top` Wrapper로 감싸 Verilator `--trace` VCD 기반 실제 Waveform · Marker는 Sampling Clock Edge

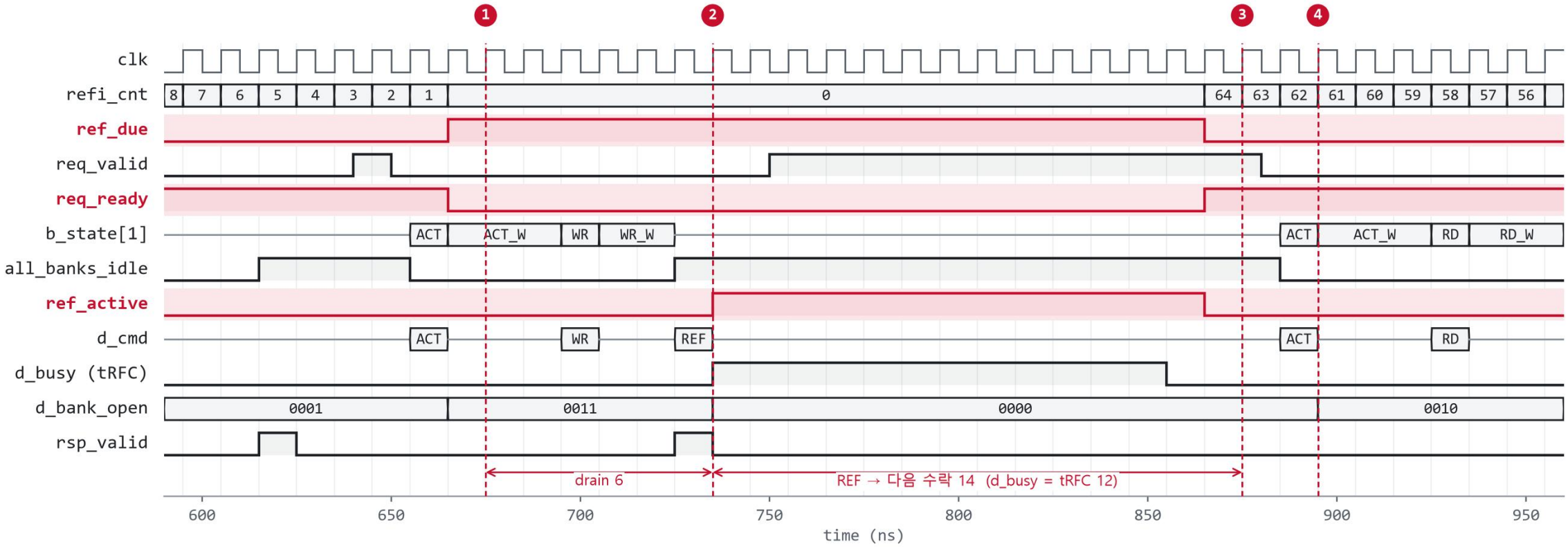
4-Bank Burst: Bank Engine 병렬 동작 및 Tag 순서 연속 Response



1. 1265 ns WR b0 r10 Accept. 2클럭 간격으로 b1 · b2 · b3 — 즉시 Dispatch되어 q_count 최대 1
2. 1325 ns b0 · b2 동시 ACT 대기 → rr_ptr = 2로 b2 우선, b0는 다음 Cycle. 132–139클럭 매 Cycle Command Issue
3. 1365 ns 첫 WR는 Closed 상태 b2 — PRE가 필요한 b0 · b1(Conflict)보다 선완료
4. 1405 ns Response는 Tag 순서 b0 → b1 → b2 → b3 연속 4클럭. 선완료 b2는 3클럭 Buffer 대기 · 총 17클럭

tb_mem_ctrl Scenario 4, 1240–1450 ns · v0는 동일 Traffic에 35클럭 · 기존 TB의 'Open Bank ≥ 2' Check는 Burst 직전 이미 0011로 병렬성 근거 불충분 (p.20 C, 이후 수정)

Refresh 동작: Queue Drain → REF → Accept 재개 및 Open Row Close



1 675 ns refi_cnt 0 → `ref_due`, 동일 Cycle부터 req_ready 0. Bank 1은 WR 진행 중

2 735 ns Bank 1 WR 완료 · All Bank IDLE · Queue Empty → REF Issue (Drain 6). `d_busy` 12클럭 = tRFC

3 855 → 875 ns busy 해제 → `ref_active` 0 · refi_cnt 64 → 대기 중이던 RD b1 r33 Accept

4 895 ns REF로 bank_open 0000 → 직전 Write한 Row 33도 ACT 필요: Hit 6이 아닌 Closed 10클럭

tb_mem_ctrl Scenario 3 구간, 590–960 ns · REF Window(Accept 차단) 20 · 14 · 14 · 9(잘림)클럭, 전체 312클럭 중 57클럭 = 18.3% (문서 추정 ≈19%)

Performance 분석: Burst Throughput 2.06× 및 Latency Trade-off

동일 TB Traffic 기준 v0 · v1 · opt(Wait Preload −1, 실험 Patch)의 Class별 Accept → Response 클럭 비교

Class (Count)	문서	v0	v1	opt
Hit RD (4)	4–5	4	6	6
Closed RD (5)	7–9	7	10	9
Closed WR (5)	6–8	6	9–12	8–10
Conflict RD (1)	10–13	11	14	12
Conflict WR (3)	9–12	10	13–14	11–12
4-Bank Burst Span	–	35	17	15

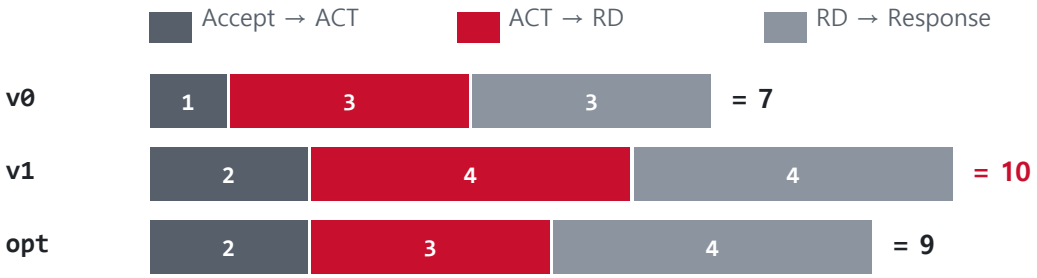
단위: 클럭 · 문서 = 분석 당시 scheduler-throughput.md §2 추정치 (v0 기준) · Write Hit는 TB 미포함

mem_ctrl.sv · 대기 타이머 preload — scratch patch · repo 미반영

```
-287  b_timer[win_bank]   <= 8'(tRP > 0 ? tRP - 1 : 0);
+287  b_timer[win_bank]   <= 8'(tRP > 1 ? tRP - 2 : 0);
-295  b_timer[win_bank]   <= 8'(tRCD > 0 ? tRCD - 1 : 0);
+295  b_timer[win_bank]   <= 8'(tRCD > 1 ? tRCD - 2 : 0);
```

- opt: tRCD · tRP 측정 Min 3 · 3 = Spec, Violation 0, 저장소 TB PASS
- Area 9,774 μm² · 2.644 ns — Overhead 미미. tRCD/tRP = 1인 경우 +1 재발생

Closed RD Latency 분해 (txn 14 · RD b0 Row 10)



+1 Queue Stage

Accept Cycle에 Queue Write, 다음 Cycle Dispatch → Engine ACT 요청. v0는 IDLE에서 즉시 Latch

+1 Wait → Issue

timer 0 Cycle에는 State만 전환, Issue는 다음 Cycle Arbiter에서 수행. t−1 Preload는 v0 기준 (opt에서 제거)

+1 Response Buffer

data_valid → `rsp_done` Register → 다음 Cycle rsp_valid. In-order 재정렬 Overhead

Throughput

4-Bank Burst 35 → 17클럭 (2.06×). 132–139클럭 구간 매 Cycle Command Issue — Command Bus가 Bottleneck. Engine 2개 이상 동시 Active 11클럭

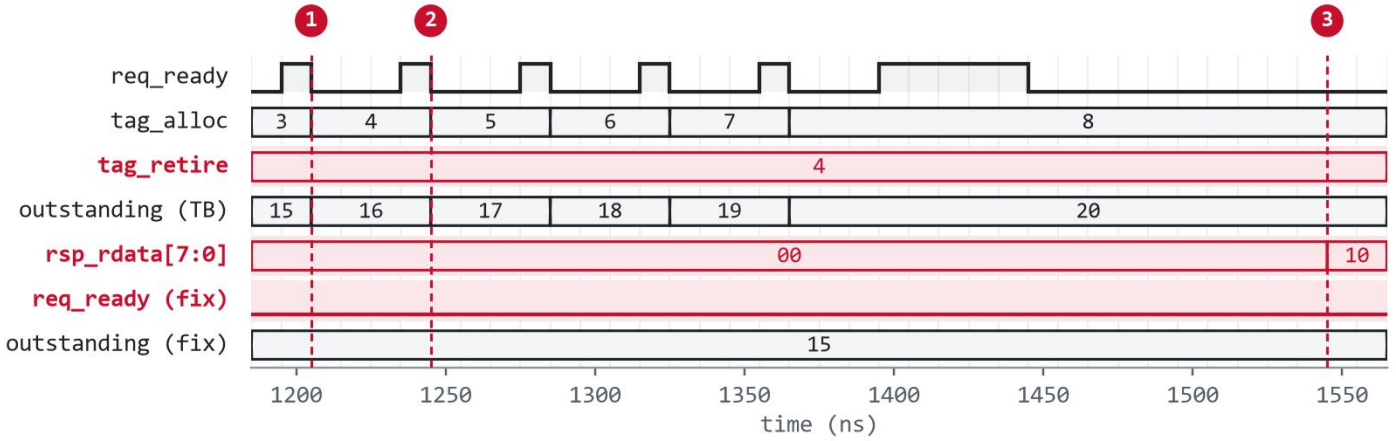
Refresh · Read Pipe

REF Window 비중 18.3% (문서 ≈19%).
RD는 `rd_inflight` 1개로 최대 4클럭당 1회 — 연속 Read Bandwidth 상한 (Back-pressure TB에서 RD 20건 간격 실측)

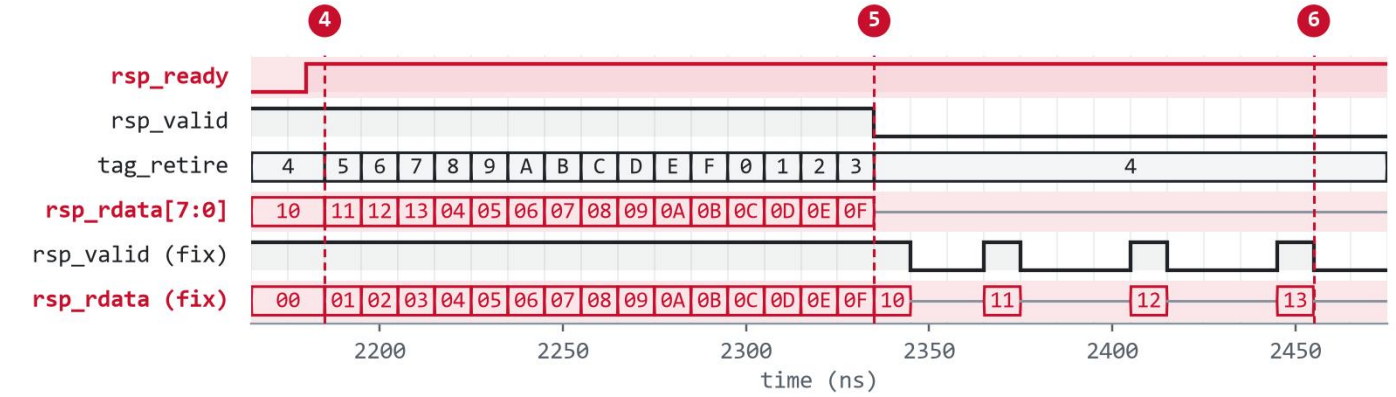
Response Back-pressure 시 Tag Wrap-around 이슈 분석 및 개선

docs/coverage.md Gap 'Backpressure'를 실험 TB로 재현 — Read 20건 연속 요청, rsp_ready 150클럭 동안 0 유지

① Accept: 미반환 Tag 4가 rd16에 재할당



② rsp_ready 해제: 선두 4건 Data 오류 · 후속 4건 Response 누락



Root Cause

`req_ready`는 Queue Free Slot만 확인 (L113).
Dispatch로 Queue가 계속 비워져
Response Block 중에도 Accept 지속,
`tag_alloc`이 16을 순환하여 미반환 `tag_retire`(4)를 재할당.
RD 완료 시 `rsp_data[b_tag]`에 Entry 점유 확인 없이 Write (L337).

1. 1205 ns Outstanding 15 → fix는 req_ready 0으로 Stall, head는 Accept 지속
2. 1245 ns rd16에 Tag 4 할당 = tag_retire (rd0 미반환)
3. 1545 ns rd16 완료 시 `rsp_data[4]`를 00 → 10으로 Overwrite
4. 2185 ns 해제 후 첫 Response: rd0 위치에 rd16 Data 10 반환
5. 2335 ns 16건 반환 후 rsp_valid 0 — rd16–19 Response 누락
6. 2455 ns fix: 00 … 13, 20건 순서대로 반환

fix · mem_ctrl.sv L113 — 저장소 반영 (commit fa08330)

```
- req_ready = free_found && !ref_due && !ref_active;
+ req_ready = free_found && !ref_due && !ref_active &&
+              (TAG_BITS'(tag_alloc + 1'b1) != tag_retire);
```

head: Response 16/20 (불일치 4 · 누락 4) → FAIL · fix: 20/20 PASS · Regression TB 추가 (7e2d4b5) · reg2reg 2.880 ns

Issue 정리: Commit History 기반 3건 및 AI 보조 재검증 개선안 5건

좌측: git log · diff · 재빌드로 확인된 변경 이력 / 우측: 재실행 · 실험 · Code Review 기반 신규 항목 (A · C는 이후 저장소 반영)

Commit History → Root Cause → 조치

Issue	Root Cause · 근거	조치
`64e73e0` Model Compile 실패	`row_match` · `legal_access` 미선언 (L107–108) — Verilator Error 2건 재현	`6049e5a`에서 Inline 처리. v0 A/B는 HEAD Model + v0 mem_ctrl로 측정
`64e73e0` access_err 도입	PRE 누락 · Illegal Row Access를 Model이 Flag, TB가 Count	현재 TB 0회. 단, 검출 경로 자체는 미검증 (E)
`6049e5a` Read Pipe 1개	Model의 CAS Data Pipe 1개 (coverage.md)	`rd_inflight`로 RD 직렬화 → RD 4클럭당 1회 (p.18)

분석 당시 문서 일부(spec §4/§5, scheduler-throughput.md)가 v0 기준 → v1 기준 갱신 완료 (d3061ca)
본 자료의 수치는 모두 v1 RTL 재실행 측정값

AI 보조 재검증 결과 (A · C 반영 완료)

A 4-bit Tag Wrap-around (p.19) — 반영 완료

Response Buffer 16 Entry 대비 Accept 상한 부재 → Overwrite · 누락. 조치: `tag_alloc + 1 ≠ tag_retire`로 Accept 제한 (fa08330)

B Wait → Issue Handoff +1 (p.18)

tRCD · tRP Spec 3, 실측 Min 4. Violation은 아니나 Class별 1–2클럭 Penalty. 제안: Preload t−2 또는 Wait 종료 Cycle Issue

C 병렬성 Check 근거 불충분 — 반영 완료

기존 'Open Bank ≥ 2' Check는 Burst 직전 0011로 이미 만족, v0도 PASS. 조치: 동시 Active Engine 수 기준 판정 (7e2d4b5)

D Queue Depth · Write Hit 미검증

Blocking Task 구조로 Queue 점유 최대 1, L259 · L271 미도달. 제안: Non-blocking 연속 요청 + 동일 Row WR

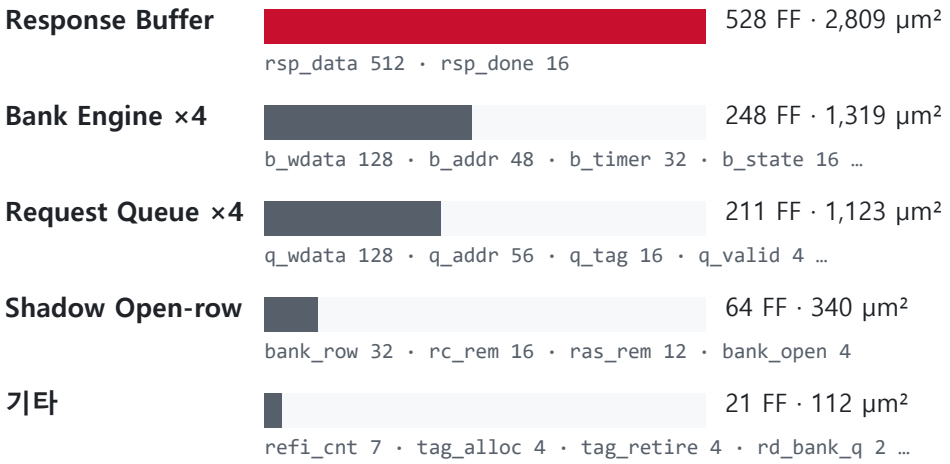
E Model Checker 검출 경로 미검증

L110 · L133 Branch 미도달 → access_err 경로 미검증. 제안: PRE 누락 · Illegal Row Negative Test, Timing Assertion

PPA 분석: Area 구성 (F/F 58%) 및 Critical Path

sv2v → Yosys 0.52 → Nangate45 typical (OpenSTA 예제 Library) → OpenSTA 3.1 · mem_ctrl 단독 · Routing · Clock Tree 미포함

F/F 1,072개 구성 (전체 DFFR/DFFS · 5.32 μm²)



Sequential 5,703 μm² = 전체 9,819의 58% · Response Buffer 단독 29% · Data 저장 F/F 768개 = 전체 F/F의 72%

Area 개선 Point (미측정 · Cell Area 기준 추정)

- Data F/F 768개는 Reset 불필요 → DFF_X1(4.522) 적용 시 약 −613 μm²
- `b_timer` 8b × 4 → 2b로 충분 (Preload 최대 2) → −24 F/F
- `b_wdata` 128 F/F는 `q_wdata`의 복사본 — Slot Index 전달 방식으로 제거 가능
- Tag 수 = Response Buffer Depth — fix의 Accept 상한과 함께 축소 시 비례 감소

	v0	v1 head	fix	opt
Area μm²	1,208	9,819	9,692	9,774
Cell	559	4,414	4,433	4,384
FF	131	1,072	1,072	1,072
reg2reg ns	0.809	2.642	2.880	2.644
≈ fmax MHz	1,182	373	343	374
Power mW *	1.63	18.26	18.31	17.94

Critical Path (reg2reg, v1 head)

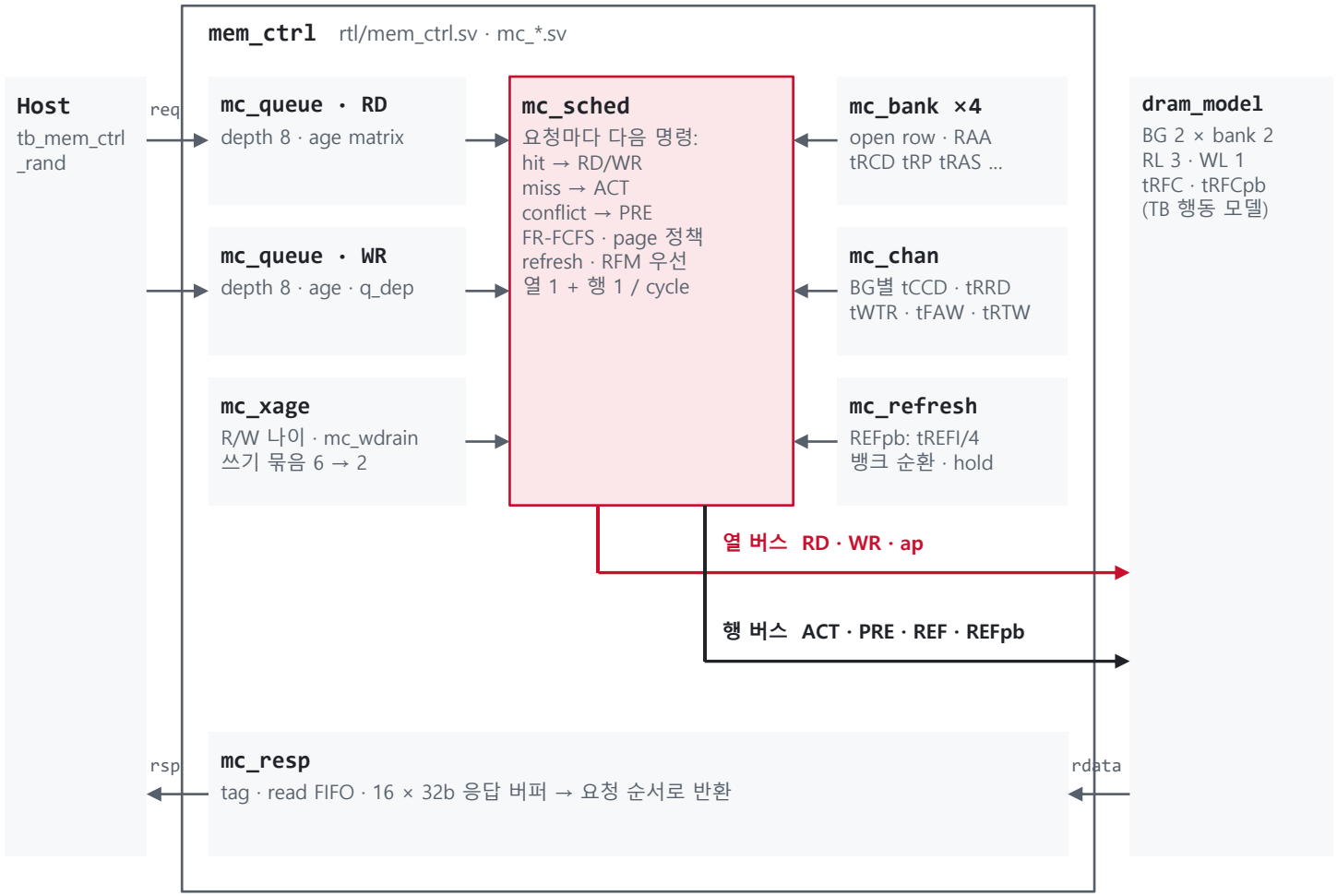
- `tag_retire[1]` → `q_tag − tag_retire` Age 비교 → Slot 선택 → Page 판정 → `b_state[2][2]` · 29 Level 2.642 ns. Fanout 46 NOR4 1단이 0.866 ns (33%)
- reg2out 1.615 ns (`rr_ptr[1]` → `dram_row[1]`, Command 출력 Register 없음) · in2reg 1.677 ns (`dram_busy`)

참고 사항

Nangate45는 공개 교육용 Library · Wire Load · Clock Tree 미포함 · *Power는 500 MHz, Input Toggle 0.1 가정 (Vector 없음), Sequential 47.6% / Combinational 52.4%. 절대값이 아닌 v0 · v1 · Patch 간 상대 비교용. 저장소 문서의 PPA는 Latency · Throughput만 다룸.

v4 구조: 두 요청 큐 → 명령 단위 스케줄러 → 행 · 열 명령 버스

요청은 RD/WR가 나갈 때까지 큐에 머물고, 스케줄러가 매 사이클 요청마다 다음 명령을 계산해 열 버스와 행 버스에 하나씩 발행



한 사이클의 결정 (요청마다)

요청 상태	다음 명령	준비 조건
hit **row 열림**	RD / WR	tRCD, 그룹별 tCCD · tWTR(RD) · tRTW(WR), 같은 주소의 더 오래된 쓰기 없음
miss **뱅크 닫힘**	ACT	tRP · tRC · tRFCpb, 그룹별 tRRD, tFAW, RAA < 12
conflict **다른 row**	PRE	tRAS · tRTP · tWR, 그 row를 쓸 요청이 두 큐에 없음

- **열 버스**: 쓰기 묶음 모드가 고른 큐의 RD/WR, 없으면 다른 큐의 RD/WR 중 가장 오래된 것
- **행 버스**: refresh(PRE → REFpb) > RFM > 고른 큐의 ACT/PRE > 다른 큐의 ACT · 쓰지 않는 row PRE > timer PRE
- **가장 오래된 것**은 age matrix에서 AND-OR 한 단. 고른 결과는 one-hot 그대로 큐 pop · tag · 데이터 선택까지 간다
- **posted write · 포워딩**: 쓰기는 수락 엣지에 응답, 쓰기 큐에 같은 주소가 있는 읽기는 DRAM 없이 응답

검증 인프라: 독립 bind 체커 + 랜덤 TB, 회귀 1,200회 위반 0

체커는 RTL과 TB를 고치지 않고 DRAM 핀만 보고 붙는다 · 같은 TB가 기본 빌드와 파라미터 변형 15개를 모두 돈다 (commit ab9d522)

DRAM 타이밍 · 프로토콜 체커 (bind)

두 명령 버스만 보고 open row · 명령 시각 · 뱅크별 refresh · RAA를 따로 재구성해 **타이밍 18종**(뱅크 그룹 S/L, tRFCpb 포함)과 **프로토콜 10종**(RAA 한도 포함)을 검사. auto-precharge는 장치가 precharge하는 엣지를 계산해 다음 ACT의 tRP를 잰다. mem_ctrl 내부 신호는 읽지 않는다.

랜덤 TB (UVM 구조를 본뜬 class TB)

sequence → driver → DUT → monitor → scoreboard(참조 메모리). 주소 매핑을 RTL과 따로 구현하고, 열 버스에 없는 row는 행 버스의 ACT로 추적. posted write는 WR가 실제로 나가는지, 포워딩은 RD가 나가지 않는지 핀에서 확인. **기능 bin 32개**, seed 재현.

호스트 SVA · directed 시나리오

valid/ready concurrent assertion 4개 + cover 5개. directed 6개: hit, miss, conflict, 4-bank burst, 응답 back-pressure(tag 16개 초과), 포워딩.

디버그: 포워딩 판정 78회 중 77회 FAIL (v3)

bind probe로 cycle 265 확인. 쓰기의 WR가 나가는 엣지에 같은 주소 읽기가 수락되는 경우를, 핀 모니터가 같은 엣지에 갱신되는 카운터를 class 핸들(vif.cyc)로 읽어 판정이 흔들린 **TB race** → TB top이 엣지 번호를 인자로 전달. RTL 변경 없이 960/960 PASS.

랜덤 회귀 (make regress)

빌드	실행	PASS	위반
기본: 두 버스 · per-bank · RFM · 쓰기 묶음 · open · XOR	300	300/300	0
refresh · 버스: bus1 · refab · bus1_refab · rfm_late	240	240/240	0
스케줄링: fcfs · fcfs_nodrain · nodrain · starve24	240	240/240	0
구조: rdpipe1 · depth4	120	120/120	0
page · 매핑: close · timer · map_brc · map_rbc · map_rcb	300	300/300	0
합계 16 빌드 · 600만 요청	1,200	1,200/1,200	0

재정렬 3,386,363 · 포워딩 151,757 · 행 / 열 동시 발행 1,359,346 · RAA 한도 도달 131,879회. 응답 순서와 데이터 모두 일치.

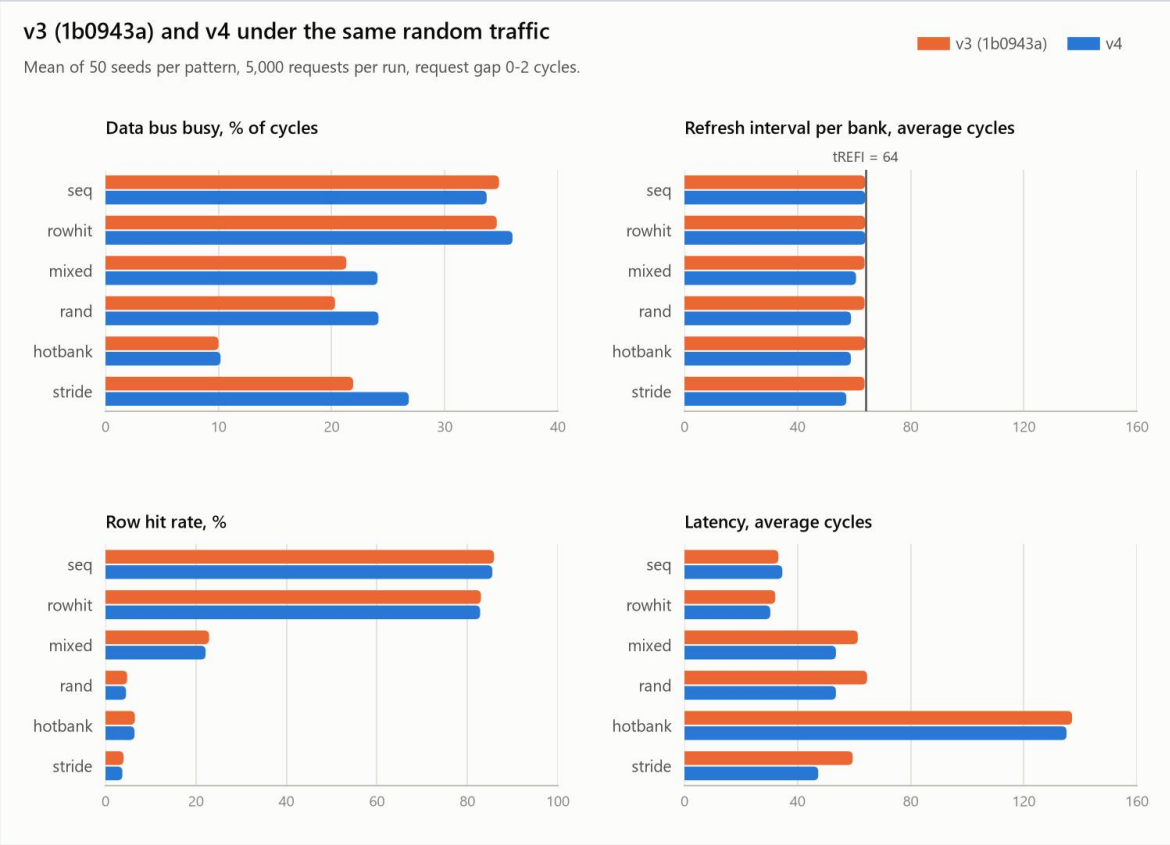
코드 커버리지 (make cov, directed → + 랜덤)

범위	Line	Branch	Toggle
컨트롤러 (mem_ctrl + mc_*)	94.8 → **98.7%**	81.8 → **98.2%**	48.5 → **93.1%**
mc_sched	91.9 → **100%**	62.5 → **96.9%**	43.2 → **91.5%**
RTL 전체 (dram_model 포함)	88.9 → **96.0%**	77.7 → **94.1%**	53.9 → **93.7%**

남은 구멍: older() 분기 2개(루프를 펼치며 상수로 접힘), 밀린 refresh 카운터 포화(설계상 미도달), WL > 1 쓰기 파이프 단, dram_model 오류 경로(negative test 필요).

성능: 같은 트래픽에서 v3 → v4, 포화 처리량을 기능별로 분해

랜덤 회귀 평균(seed 50 × 5,000건)과 포화(요청 간격 0, seed 1) · 표의 가운데 네 열은 v4를 파라미터로 바꾼 빌드



v3 = commit 1b0943a(뱅크 그룹 없는 장치), v4 = ab9d522. conflict가 많은 트래픽이 가장 많이 올랐다(rand +19%, stride +22%). seq만 −3%: 같은 그룹 안 연속 RD/WR가 tCCD_L(2)를 지켜야 한다. 뱅크당 refresh 간격 57–64(RFM의 REFpb 포함), 밀린 refresh 최대 1개.

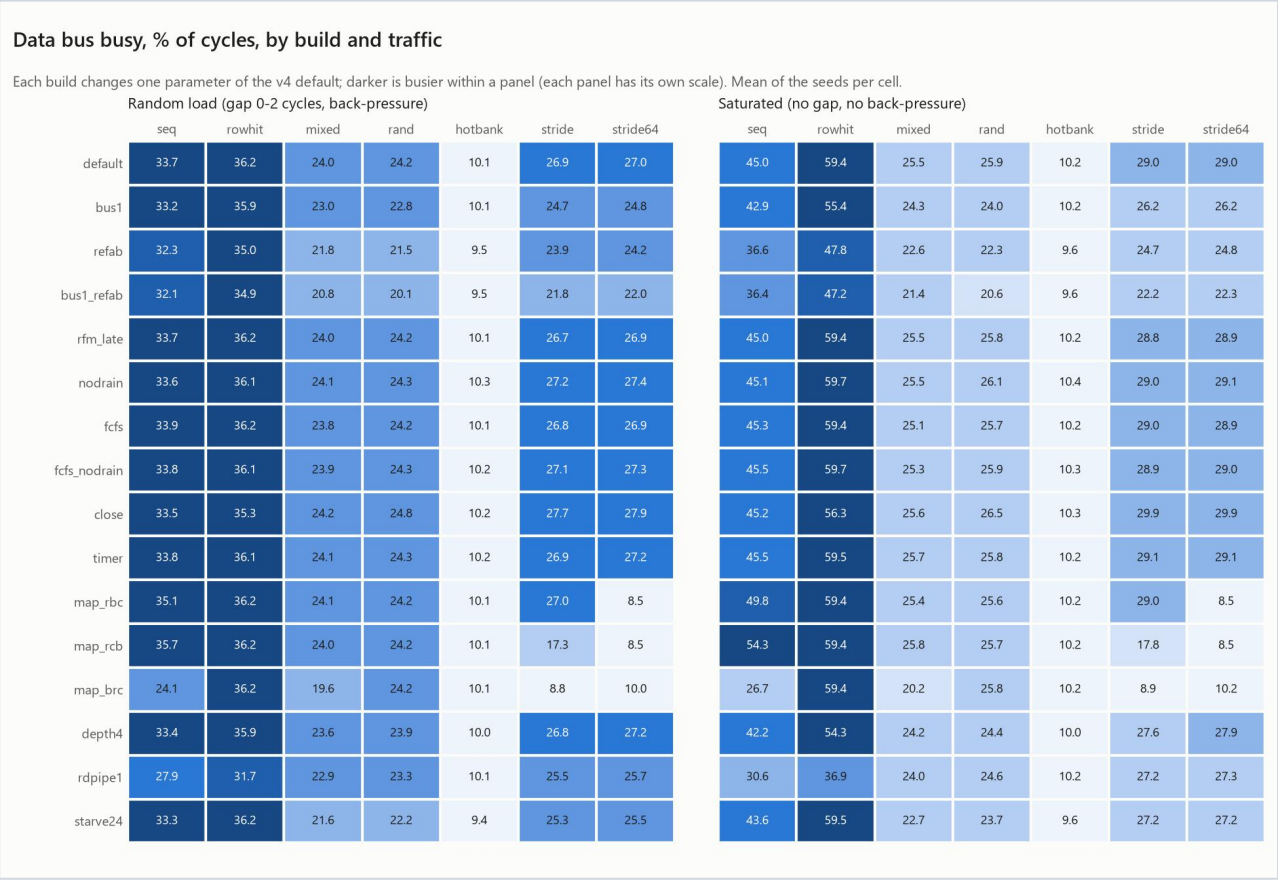
포화 처리량: 두 기능을 하나씩 (데이터 버스 사용률)

트래픽	v3 방식 bus1_refab	버스 분리만 refab	per-bank만 bus1	v4	v4 + row:col:bank
row hit, 읽기만	63.3%	65.6%	71.8%	**80.7%**	80.7%
row hit, 쓰기만	60.4%	62.1%	72.3%	**80.7%**	80.7%
row hit, 반반	47.3%	47.8%	55.2%	**59.6%**	59.6%
연속 주소, 읽기만	39.7%	40.2%	48.0%	**50.0%**	**78.5%**
랜덤 주소, 읽기만	21.9%	23.4%	24.7%	**26.5%**	27.1%
패턴 혼합, 반반	21.7%	22.7%	24.3%	**25.7%**	25.6%

- **per-bank refresh가 가장 크다**: row hit 읽기 63.3 → 71.8%. all-bank는 tREFI마다 모든 row를 닫고 약 25사이클 멈춘다
- **두 기능은 서로 키운다**: 버스 분리 단독은 +1–7%, per-bank와 함께면 71.8 → 80.7%. refresh의 PRE · REFpb가 행 버스로 가서 RD 흐름을 끊지 않는다
- **뱅크 그룹은 매핑을 바꾼다**: 연속 16 word가 한 그룹에 있는 XOR은 연속 읽기가 50%에서 막히고, 두 word마다 그룹을 바꾸는 row:col:bank는 78.5%
- **경로별 latency**: hit RD 5 · miss RD 8 · conflict RD 11사이클, posted WR와 포워딩된 RD는 1사이클 (v3부터)

트래픽 × 기능 분석: 빌드 16 × 트래픽 7 × 두 부하 = 1,456회

v4 기본에서 파라미터를 하나씩 바꾼 빌드를 같은 트래픽에 돌리고 같은 파라미터로 합성 (make matrix · ppa-sweep)



데이터 버스 사용률(%). 왼쪽 랜덤 부하(seed 10), 오른쪽 포화(seed 3). 1,456회 모두 PASS, 규칙 위반 0, 데이터 불일치 0 (RTL ab9d522, 스크립트 322632f).

질문별 답 (포화 평균 기준)

질문	답	근거
가장 큰 손잡이	**per-bank refresh**	all-bank로 바꾸면 –16%, 면적 차이 2%
명령 버스 분리	면적 비용 없음	하나로 바꾸면 –7%, 면적 0.4% 이내
주소 매핑	**XOR만** 모든 stride에서 버팀	64 word stride: XOR 27.0%, 나머지 8.5–10.0%
큐 8 + 8	처리량 +6%에 면적 +46%	4 + 4는 읽기 latency 절반, 면적 –31%
RD 파이프라이닝	필수	RD 1개로 줄이면 –19%, 면적 0.4%
FR-FCFS vs FCFS	이 모델에서는 같음	FCFS가 면적 +8% (두 큐 사이 나이 필요)
쓰기 모아 보내기	평균 ↓, 꼬리 ↑	읽기 평균 –1.7%, p99 +9%, 방향 전환 –19%
RFM 미리 주기	빼도 된다	성능 같음, 면적 –0.8%

fmax 차이는 ABC 잡음(±12%) 안이면 구조 차이로 보지 않는다: 비트 배선만 다른 세 매핑이 모두 310 MHz, XOR 게이트 두 개가 더 있는 기본 빌드가 348 MHz. FF 수는 정확하고 면적은 ±2–3% 안에서 믿을 만하다.

PPA: 기능은 늘고 fmax는 유지 — 버전별 면적 · 타이밍과 기능별 비용

sv2v → Yosys 0.52 → Nangate45 typical → OpenSTA 3.1 · mem_ctrl 단독(배선 · 클럭 트리 없음) · 버전마다 그 commit을 다시 합성

버전별 (make ppa)

	v1 fa08330	v2 c1b8248	v3 첫 구현 커밋 전	v3 1b0943a	v4 ab9d522
Area μm²	9,692	13,478	19,263	18,600	**18,912**
Cell	4,433	6,781	10,991	9,924	**9,995**
FF	1,072	1,344	1,582	1,582	**1,616**
reg2reg ns	2.880	2.753	4.765	2.922	**2.831**
≈ fmax MHz	343	357	208	337	**348**

v3 타이밍 클로저: 208 → 264 → 337 MHz

- **증상**: 쓰기 모드 → 후보 선택 → 큐 pop → 같은 주소 비교 → 포워딩 · req_ready → push. fanout 58 · 104 넷 두 개가 2 ns
- **① 수락 경로 분리**: 같은 주소 비교가 이번 엣지에 나가는 엔트리도 그대로 비교 → 스케줄러 결정을 기다리지 않음
- **② 후보 집합 통합**: "고른 큐, 없으면 다른 큐" 한 집합씩으로 가장 오래된 것 고르기 5 → 3번, pop 때 의존 비트 지우기 제거
- **③ one-hot 유지**: 인덱스로 바꿨다 다시 푸는 단계(fanout 72) 제거, AND-OR 선택
- **v4**: 두 버스라 RD/WR와 ACT/PRE 중 고르는 단계가 사라짐 → RFM을 넣고도 348 MHz (RFM 전 377 MHz)

기능 하나를 뺐을 때 (포화 평균 · 랜덤 부하 읽기 latency)

기능 (대신)	빼면 처리량	빼면 latency	이 기능의 면적	판단
per-bank refresh (all-bank)	−16%	+10%	+2%	가장 싸고 가장 큼
두 명령 버스 (하나)	−7%	+4%	~0%	공짜에 가까움
RD 4개 in flight (1개)	−19%	+8%	+0.4%	필수
큐 8 + 8 (4 + 4)	−6%	−45%	+46%	처리량 우선일 때만
XOR 매핑 (row:bank:col)	stride64 −69%	+28%	잡음 범위	기본 유지
FR-FCFS (FCFS)	0%	0%	−8%	이 모델에선 면적 이득
쓰기 묶음 (오래된 순)	0%	+2%	−2%	평균 · 전환 vs p99
미리 RFM (한도에서만)	0%	0%	+0.8%	빼도 된다

"빼면"은 기본 빌드 대비 그 변형 빌드의 변화, 면적은 그 기능이 기본 빌드에 더하는 양이다. fmax는 잡음(±12%)보다 큰 차이만 의미가 있다(depth4 +18%, timer −23%, fcfs_nodrain −24%). 권장: 범용은 지금의 기본값, 면적 · latency 우선이면 큐 4 + 4(처리량 −6%, 면적 −31%). ≈ fmax = 1 / 최소 주기(reg2reg + 플롭 setup, v4는 2.831 + 0.043 = 2.874 ns).

12 Conclusion

핵심 요약 및 향후 계획

Spec → RTL → 검증 → 성능 / PPA 분석 전 과정 결과와 미해결 항목 정리

핵심 요약

같은 DRAM 모델에서 컨트롤러를 다섯 번 설계하며 전환마다 성능과 비용을 수치로 비교하고,
독립 체커와 랜덤 회귀(1,200회, 위반 0)로 단계마다 정확성을 확인

향후 계획

- 뱅크 그룹을 word 단위로 섞는 XOR 매핑 (연속 주소와 stride를 함께)
- 배치 · 배선 흐름(OpenROAD)으로 기능별 타이밍 비용 재측정
- 응답 버퍼(FF 34%) 축소, dram_model 오류 경로 negative test
- DFI 1 : 2(400 MHz, 클럭당 명령 2개)와 RD_PIPE를 PHY 왕복에 맞추기 (HSL 연결에서 나온 과제)
- 실제 장치 규모(pseudo channel · 뱅크 16개 이상)에서 refresh · RFM 비중 재평가

권용현

010-6408-8646

kyongh0331@naver.com

github.com/yhkwon-git/mini-dram-controller